
Problems
7-1 Hoare partition correctness

The version of PARTITION given in this chapter is not the original partitioning algorithm. Here is the original partitioning algorithm, which is due to C. A. R. Hoare.

```

HOARE-PARTITION( $A, p, r$ )
1   $x = A[p]$ 
2   $i = p - 1$ 
3   $j = r + 1$ 
4  while TRUE
5      repeat
6           $j = j - 1$ 
7      until  $A[j] \leq x$ 
8      repeat
9           $i = i + 1$ 
10     until  $A[i] \geq x$ 
11     if  $i < j$ 
12         exchange  $A[i]$  with  $A[j]$ 
13     else return  $j$ 

```

- a. Demonstrate the operation of HOARE-PARTITION on the array $A = \langle 13, 19, 9, 5, 12, 8, 7, 4, 11, 2, 6, 21 \rangle$, showing the values of the array and the indices i and j after each iteration of the **while** loop in lines 4–13.
- b. Describe how the PARTITION procedure in Section 7.1 differs from HOARE-PARTITION when all elements in $A[p : r]$ are equal. Describe a practical advantage of HOARE-PARTITION over PARTITION for use in quicksort.

The next three questions ask you to give a careful argument that the procedure HOARE-PARTITION is correct. Assuming that the subarray $A[p : r]$ contains at least two elements, prove the following:

- c. The indices i and j are such that the procedure never accesses an element of A outside the subarray $A[p : r]$.
- d. When HOARE-PARTITION terminates, it returns a value j such that $p \leq j < r$.
- e. Every element of $A[p : j]$ is less than or equal to every element of $A[j + 1 : r]$ when HOARE-PARTITION terminates.

The PARTITION procedure in Section 7.1 separates the pivot value (originally in $A[r]$) from the two partitions it forms. The HOARE-PARTITION procedure, on the other hand, always places the pivot value (originally in $A[p]$) into one of the two partitions $A[p:j]$ and $A[j+1:r]$. Since $p \leq j < r$, neither partition is empty.

f. Rewrite the QUICKSORT procedure to use HOARE-PARTITION.

7-2 Quicksort with equal element values

The analysis of the expected running time of randomized quicksort in Section 7.4.2 assumes that all element values are distinct. This problem examines what happens when they are not.

- a.* Suppose that all element values are equal. What is randomized quicksort's running time in this case?
- b.* The PARTITION procedure returns an index q such that each element of $A[p:q-1]$ is less than or equal to $A[q]$ and each element of $A[q+1:r]$ is greater than $A[q]$. Modify the PARTITION procedure to produce a procedure $\text{PARTITION}'(A, p, r)$, which permutes the elements of $A[p:r]$ and returns two indices q and t , where $p \leq q \leq t \leq r$, such that
 - all elements of $A[q:t]$ are equal,
 - each element of $A[p:q-1]$ is less than $A[q]$, and
 - each element of $A[t+1:r]$ is greater than $A[q]$.

Like PARTITION, your $\text{PARTITION}'$ procedure should take $\Theta(r-p)$ time.

- c.* Modify the RANDOMIZED-PARTITION procedure to call $\text{PARTITION}'$, and name the new procedure $\text{RANDOMIZED-PARTITION}'$. Then modify the QUICKSORT procedure to produce a procedure $\text{QUICKSORT}'(A, p, r)$ that calls $\text{RANDOMIZED-PARTITION}'$ and recurses only on partitions where elements are not known to be equal to each other.
- d.* Using $\text{QUICKSORT}'$, adjust the analysis in Section 7.4.2 to avoid the assumption that all elements are distinct.

7-3 Alternative quicksort analysis

An alternative analysis of the running time of randomized quicksort focuses on the expected running time of each individual recursive call to $\text{RANDOMIZED-QUICKSORT}$, rather than on the number of comparisons performed. As in the analysis of Section 7.4.2, assume that the values of the elements are distinct.

- a.* Argue that, given an array of size n , the probability that any particular element is chosen as the pivot is $1/n$. Use this probability to define indicator random variables $X_i = I\{i\text{th smallest element is chosen as the pivot}\}$. What is $E[X_i]$?
- b.* Let $T(n)$ be a random variable denoting the running time of quicksort on an array of size n . Argue that

$$E[T(n)] = E\left[\sum_{q=1}^n X_q (T(q-1) + T(n-q) + \Theta(n))\right]. \quad (7.2)$$

- c.* Show how to rewrite equation (7.2) as

$$E[T(n)] = \frac{2}{n} \sum_{q=1}^{n-1} E[T(q)] + \Theta(n). \quad (7.3)$$

- d.* Show that

$$\sum_{q=1}^{n-1} q \lg q \leq \frac{n^2}{2} \lg n - \frac{n^2}{8} \quad (7.4)$$

for $n \geq 2$. (*Hint:* Split the summation into two parts, one summation for $q = 1, 2, \dots, \lceil n/2 \rceil - 1$ and one summation for $q = \lceil n/2 \rceil, \dots, n-1$.)

- e.* Using the bound from equation (7.4), show that the recurrence in equation (7.3) has the solution $E[T(n)] = O(n \lg n)$. (*Hint:* Show, by substitution, that $E[T(n)] \leq an \lg n$ for sufficiently large n and for some positive constant a .)

7-4 Stooge sort

Professors Howard, Fine, and Howard have proposed a deceptively simple sorting algorithm, named stooge sort in their honor, appearing on the following page.

- a.* Argue that the call `STOOGESORT( $A, 1, n$ )` correctly sorts the array $A[1:n]$.
- b.* Give a recurrence for the worst-case running time of `STOOGESORT` and a tight asymptotic (Θ -notation) bound on the worst-case running time.
- c.* Compare the worst-case running time of `STOOGESORT` with that of insertion sort, merge sort, heapsort, and quicksort. Do the professors deserve tenure?

```

STOOGESORT( $A, p, r$ )
1  if  $A[p] > A[r]$ 
2      exchange  $A[p]$  with  $A[r]$ 
3  if  $p + 1 < r$ 
4       $k = \lfloor (r - p + 1)/3 \rfloor$            // round down
5      STOOGESORT( $A, p, r - k$ )         // first two-thirds
6      STOOGESORT( $A, p + k, r$ )         // last two-thirds
7      STOOGESORT( $A, p, r - k$ )         // first two-thirds again

```

7-5 Stack depth for quicksort

The QUICKSORT procedure of Section 7.1 makes two recursive calls to itself. After QUICKSORT calls PARTITION, it recursively sorts the low side of the partition and then it recursively sorts the high side of the partition. The second recursive call in QUICKSORT is not really necessary, because the procedure can instead use an iterative control structure. This transformation technique, called *tail-recursion elimination*, is provided automatically by good compilers. Applying tail-recursion elimination transforms QUICKSORT into the TRE-QUICKSORT procedure.

```

TRE-QUICKSORT( $A, p, r$ )
1  while  $p < r$ 
2      // Partition and then sort the low side.
3       $q = \text{PARTITION}(A, p, r)$ 
4      TRE-QUICKSORT( $A, p, q - 1$ )
5       $p = q + 1$ 

```

a. Argue that TRE-QUICKSORT($A, 1, n$) correctly sorts the array $A[1 : n]$.

Compilers usually execute recursive procedures by using a *stack* that contains pertinent information, including the parameter values, for each recursive call. The information for the most recent call is at the top of the stack, and the information for the initial call is at the bottom. When a procedure is called, its information is *pushed* onto the stack, and when it terminates, its information is *popped*. Since we assume that array parameters are represented by pointers, the information for each procedure call on the stack requires $O(1)$ stack space. The *stack depth* is the maximum amount of stack space used at any time during a computation.

b. Describe a scenario in which TRE-QUICKSORT's stack depth is $\Theta(n)$ on an n -element input array.

- c. Modify TRE-QUICKSORT so that the worst-case stack depth is $\Theta(\lg n)$. Maintain the $O(n \lg n)$ expected running time of the algorithm.

7-6 Median-of-3 partition

One way to improve the RANDOMIZED-QUICKSORT procedure is to partition around a pivot that is chosen more carefully than by picking a random element from the subarray. A common approach is the *median-of-3* method: choose the pivot as the median (middle element) of a set of 3 elements randomly selected from the subarray. (See Exercise 7.4-6.) For this problem, assume that the n elements in the input subarray $A[p:r]$ are distinct and that $n \geq 3$. Denote the sorted version of $A[p:r]$ by $z_1, z_2, \dots, z_n$. Using the median-of-3 method to choose the pivot element x , define $p_i = \Pr\{x = z_i\}$.

- a. Give an exact formula for p_i as a function of n and i for $i = 2, 3, \dots, n-1$. (Observe that $p_1 = p_n = 0$.)
- b. By what amount does the median-of-3 method increase the likelihood of choosing the pivot to be $x = z_{\lfloor (n+1)/2 \rfloor}$, the median of $A[p:r]$, compared with the ordinary implementation? Assume that $n \rightarrow \infty$, and give the limiting ratio of these probabilities.
- c. Suppose that we define a “good” split to mean choosing the pivot as $x = z_i$, where $n/3 \leq i \leq 2n/3$. By what amount does the median-of-3 method increase the likelihood of getting a good split compared with the ordinary implementation? (*Hint*: Approximate the sum by an integral.)
- d. Argue that in the $\Omega(n \lg n)$ running time of quicksort, the median-of-3 method affects only the constant factor.

7-7 Fuzzy sorting of intervals

Consider a sorting problem in which you do not know the numbers exactly. Instead, for each number, you know an interval on the real line to which it belongs. That is, you are given n closed intervals of the form $[a_i, b_i]$, where $a_i \leq b_i$. The goal is to *fuzzy-sort* these intervals: to produce a permutation $\langle i_1, i_2, \dots, i_n \rangle$ of the intervals such that for $j = 1, 2, \dots, n$, there exist $c_j \in [a_{i_j}, b_{i_j}]$ satisfying $c_1 \leq c_2 \leq \dots \leq c_n$.

- a. Design a randomized algorithm for fuzzy-sorting n intervals. Your algorithm should have the general structure of an algorithm that quicksorts the left endpoints (the a_i values), but it should take advantage of overlapping intervals to improve the running time. (As the intervals overlap more and more, the prob-

lem of fuzzy-sorting the intervals becomes progressively easier. Your algorithm should take advantage of such overlapping, to the extent that it exists.)

- b.* Argue that your algorithm runs in $\Theta(n \lg n)$ expected time in general, but runs in $\Theta(n)$ expected time when all of the intervals overlap (i.e., when there exists a value x such that $x \in [a_i, b_i]$ for all i). Your algorithm should not be checking for this case explicitly, but rather, its performance should naturally improve as the amount of overlap increases.

Chapter notes

Quicksort was invented by Hoare [219], and his version of PARTITION appears in Problem 7-1. Bentley [51, p. 117] attributes the PARTITION procedure given in Section 7.1 to N. Lomuto. The analysis in Section 7.4 based on an analysis due to Motwani and Raghavan [336]. Sedgewick [401] and Bentley [51] provide good references on the details of implementation and how they matter.

McIlroy [323] shows how to engineer a “killer adversary” that produces an array on which virtually any implementation of quicksort takes $\Theta(n^2)$ time.

8 Sorting in Linear Time

We have now seen a handful of algorithms that can sort n numbers in $O(n \lg n)$ time. Whereas merge sort and heapsort achieve this upper bound in the worst case, quicksort achieves it on average. Moreover, for each of these algorithms, we can produce a sequence of n input numbers that causes the algorithm to run in $\Omega(n \lg n)$ time.

These algorithms share an interesting property: *the sorted order they determine is based only on comparisons between the input elements*. We call such sorting algorithms **comparison sorts**. All the sorting algorithms introduced thus far are comparison sorts.

In Section 8.1, we'll prove that any comparison sort must make $\Omega(n \lg n)$ comparisons in the worst case to sort n elements. Thus, merge sort and heapsort are asymptotically optimal, and no comparison sort exists that is faster by more than a constant factor.

Sections 8.2, 8.3, and 8.4 examine three sorting algorithms—counting sort, radix sort, and bucket sort—that run in linear time on certain types of inputs. Of course, these algorithms use operations other than comparisons to determine the sorted order. Consequently, the $\Omega(n \lg n)$ lower bound does not apply to them.

8.1 Lower bounds for sorting

A comparison sort uses only comparisons between elements to gain order information about an input sequence $\langle a_1, a_2, \dots, a_n \rangle$. That is, given two elements a_i and a_j , it performs one of the tests $a_i < a_j$, $a_i \leq a_j$, $a_i = a_j$, $a_i \geq a_j$, or $a_i > a_j$ to determine their relative order. It may not inspect the values of the elements or gain order information about them in any other way.

Since we are proving a lower bound, we assume without loss of generality in this section that all the input elements are distinct. After all, a lower bound for distinct elements applies when elements may or may not be distinct. Consequently,

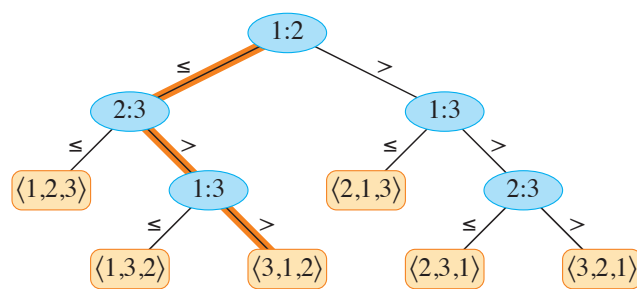


Figure 8.1 The decision tree for insertion sort operating on three elements. An internal node (shown in blue) annotated by $i:j$ indicates a comparison between a_i and a_j . A leaf annotated by the permutation $\langle \pi(1), \pi(2), \pi(n) \rangle$ indicates the ordering $a_{\pi(1)} \leq a_{\pi(2)} \leq \dots \leq a_{\pi(n)}$. The highlighted path indicates the decisions made when sorting the input sequence $\langle a_1 = 6, a_2 = 8, a_3 = 5 \rangle$. Going left from the root node, labeled 1:2, indicates that $a_1 \leq a_2$. Going right from the node labeled 2:3 indicates that $a_2 > a_3$. Going right from the node labeled 1:3 indicates that $a_1 > a_3$. Therefore, we have the ordering $a_3 \leq a_1 \leq a_2$, as indicated in the leaf labeled $\langle 3, 1, 2 \rangle$. Because the three input elements have $3! = 6$ possible permutations, the decision tree must have at least 6 leaves.

comparisons of the form $a_i = a_j$ are useless, which means that we can assume that no comparisons for exact equality occur. Moreover, the comparisons $a_i \leq a_j$, $a_i \geq a_j$, $a_i > a_j$, and $a_i < a_j$ are all equivalent in that they yield identical information about the relative order of a_i and a_j . We therefore assume that all comparisons have the form $a_i \leq a_j$.

The decision-tree model

We can view comparison sorts abstractly in terms of decision trees. A **decision tree** is a full binary tree (each node is either a leaf or has both children) that represents the comparisons between elements that are performed by a particular sorting algorithm operating on an input of a given size. Control, data movement, and all other aspects of the algorithm are ignored. Figure 8.1 shows the decision tree corresponding to the insertion sort algorithm from Section 2.1 operating on an input sequence of three elements.

A decision tree has each internal node annotated by $i:j$ for some i and j in the range $1 \leq i, j \leq n$, where n is the number of elements in the input sequence. We also annotate each leaf by a permutation $\langle \pi(1), \pi(2), \pi(n) \rangle$. (See Section C.1 for background on permutations.) Indices in the internal nodes and the leaves always refer to the original positions of the array elements at the start of the sorting algorithm. The execution of the comparison sorting algorithm corresponds to tracing a simple path from the root of the decision tree down to a leaf. Each internal node indicates a comparison $a_i \leq a_j$. The left subtree then dictates sub-

sequent comparisons once we know that $a_i \leq a_j$, and the right subtree dictates subsequent comparisons when $a_i > a_j$. Arriving at a leaf, the sorting algorithm has established the ordering $a_{\pi(1)} \leq a_{\pi(2)} \leq \cdots \leq a_{\pi(n)}$. Because any correct sorting algorithm must be able to produce each permutation of its input, each of the $n!$ permutations on n elements must appear as at least one of the leaves of the decision tree for a comparison sort to be correct. Furthermore, each of these leaves must be reachable from the root by a downward path corresponding to an actual execution of the comparison sort. (We call such leaves “reachable.”) Thus, we consider only decision trees in which each permutation appears as a reachable leaf.

A lower bound for the worst case

The length of the longest simple path from the root of a decision tree to any of its reachable leaves represents the worst-case number of comparisons that the corresponding sorting algorithm performs. Consequently, the worst-case number of comparisons for a given comparison sort algorithm equals the height of its decision tree. A lower bound on the heights of all decision trees in which each permutation appears as a reachable leaf is therefore a lower bound on the running time of any comparison sort algorithm. The following theorem establishes such a lower bound.

Theorem 8.1

Any comparison sort algorithm requires $\Omega(n \lg n)$ comparisons in the worst case.

Proof From the preceding discussion, it suffices to determine the height of a decision tree in which each permutation appears as a reachable leaf. Consider a decision tree of height h with l reachable leaves corresponding to a comparison sort on n elements. Because each of the $n!$ permutations of the input appears as one or more leaves, we have $n! \leq l$. Since a binary tree of height h has no more than 2^h leaves, we have

$$n! \leq l \leq 2^h,$$

which, by taking logarithms, implies

$$\begin{aligned} h &\geq \lg(n!) && \text{(since the } \lg \text{ function is monotonically increasing)} \\ &= \Omega(n \lg n) && \text{(by equation (3.28) on page 67).} \end{aligned}$$

■

Corollary 8.2

Heapsort and merge sort are asymptotically optimal comparison sorts.

Proof The $O(n \lg n)$ upper bounds on the running times for heapsort and merge sort match the $\Omega(n \lg n)$ worst-case lower bound from Theorem 8.1. ■

Exercises**8.1-1**

What is the smallest possible depth of a leaf in a decision tree for a comparison sort?

8.1-2

Obtain asymptotically tight bounds on $\lg(n!)$ without using Stirling's approximation. Instead, evaluate the summation $\sum_{k=1}^n \lg k$ using techniques from Section A.2.

8.1-3

Show that there is no comparison sort whose running time is linear for at least half of the $n!$ inputs of length n . What about a fraction of $1/n$ of the inputs of length n ? What about a fraction $1/2^n$?

8.1-4

You are given an n -element input sequence, and you know in advance that it is partly sorted in the following sense. Each element initially in position i such that $i \bmod 4 = 0$ is either already in its correct position, or it is one place away from its correct position. For example, you know that after sorting, the element initially in position 12 belongs in position 11, 12, or 13. You have no advance information about the other elements, in positions i where $i \bmod 4 \neq 0$. Show that an $\Omega(n \lg n)$ lower bound on comparison-based sorting still holds in this case.

8.2 Counting sort

Counting sort assumes that each of the n input elements is an integer in the range 0 to k , for some integer k . It runs in $\Theta(n + k)$ time, so that when $k = O(n)$, counting sort runs in $\Theta(n)$ time.

Counting sort first determines, for each input element x , the number of elements less than or equal to x . It then uses this information to place element x directly into its position in the output array. For example, if 17 elements are less than or equal to x , then x belongs in output position 17. We must modify this scheme slightly to handle the situation in which several elements have the same value, since we do not want them all to end up in the same position.

The COUNTING-SORT procedure on the facing page takes as input an array $A[1:n]$, the size n of this array, and the limit k on the nonnegative integer values in A . It returns its sorted output in the array $B[1:n]$ and uses an array $C[0:k]$ for temporary working storage.